

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4

По дисциплине «Методы обработки изображений»
«Формирование изображения методом трассировки путей»

Выполнил:

Студент группы Р3306,
Михайлов Дмитрий Андреевич

Преподаватель:

Жданов Дмитрий Дмитриевич



Санкт-Петербург

2026 год

Содержание

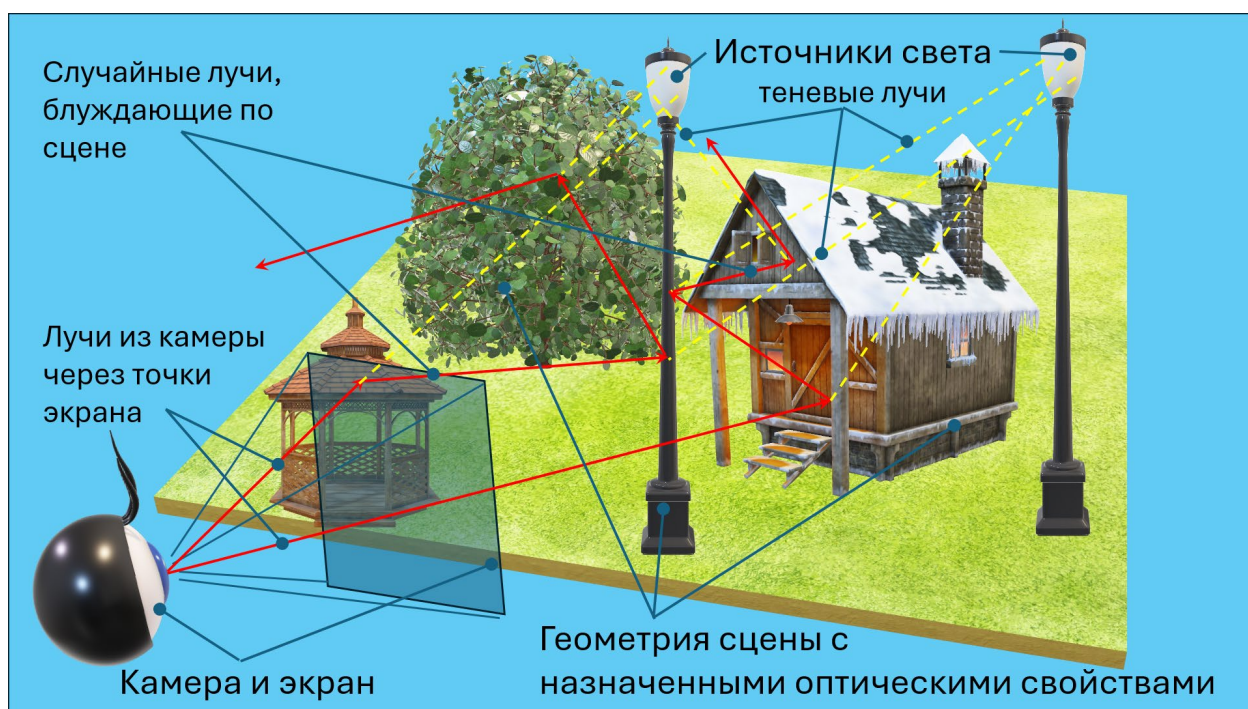
Цель.....	3
Задание	3
Исходные данные и допущения	3
Описание алгоритмов и методов, используемых в программе.....	4
Описание структуры программы и основных классов и функций.....	10
Подробное пояснение ключевых фрагментов программы	23
Тестирование программы и примеры полученных результатов.....	33
Заключение	38

Цель

Освоить простейшие методы синтеза изображений трехмерных сцен с учетом глобального освещения методом трассировки путей. Убедиться и доказать корректность сформированного изображения.

Задание

Построить изображение трехмерной сцены с корректным учетом глобального освещения методом трассировки путей.



Исходные данные и допущения

- Модель геометрии сцены, формируется треугольной сеткой. Треугольная сетка может быть сформирована вручную или импортирована из OBJ файла. В любом случае необходимо понимать как модель будет выглядеть на изображении. Для поиска точки встречи луча с поверхностью можно написать свою функцию, можно воспользоваться библиотекой Embree (или любой другой библиотекой).
- Цветовая модель – RGB (можно спектральная).
- Модель оптических свойств материалов. Пропускание отсутствует, среда равномерная и прозрачная с постоянным показателем преломления $n = 1$. Оптические свойства на поверхностях: коэффициент диффузного отражения (Ламберт) и коэффициент зеркального отражения (чистое зеркало). Зеркальные и диффузные оптические свойства могут быть заданы одновременно на одной поверхности. Выбирается одно событие (свойство) методами выборки по значимости и русской рулетки. Коэффициенты цветные. Новое направление и цвет луча выбираются в соответствии с выбранным событием. Желательно обеспечить постоянство энергии луча. Более сложные модели (например, Кука-Торранса) возможны, но не обязательны. Обязательное условие – физичность модели, т.е. суммарное отражение для каждой компоненты цвета не превышает 1.
- Камера точечная (можно использовать линзовый объектив, например идеальную тонкую линзу), смотрит и видит сцену. Разрешение до 1000x1000 точек, но не менее 500x500.

Антиалиасинг обеспечивается случайным выбором координат точки в пределах одного пикселя.

- Источники света – протяженные, диаграмма излучения – Ламберт. Источники цветные. Расчет прямого освещения можно ограничить одним случайным лучом со случайно выбранного источника света. Использовать выборку по значимости в соответствии с мощностью источника света, току на источнике выбирать случайным образом, Вариация яркости по поверхности треугольника отсутствует.
- Продолжительность расчета можно задать временем, точностью или числом лучей, выпущенных с каждого пикселя изображения.
- Изображение формируется в абсолютных яркостях и затем переводится в область относительных яркостей с нормировкой на некоторое значение (например, максимальную яркость, среднюю яркость или заранее заданный уровень). Все значения, превышающие 1, отсекаются до уровня 1. Далее применяется гамма-коррекция по формуле $V_{corr} = V^{(1/\gamma)}$, обычно $\gamma = 2.2$; после этого результат приводится к диапазону 0–255 и записывается в файл изображения. В программе используется простейший формат PPM. Возможен вариант записи изображения в реальных яркостях (например, HDR) с последующей визуализацией результата.

Результат представит в виде отчета, с описанием алгоритмов и методов, используемых в программе, текста программы (с пониманием текста), и файлов изображений. Программа должна иметь достаточно простой интерфейс (любого вида) для возможности изменить параметры сцены и получить новый результат в процессе защиты работы.

Описание алгоритмов и методов, используемых в программе

1. Общая идея метода

В данной лабораторной работе изображение трёхмерной сцены формируется методом трассировки путей. Суть метода состоит в моделировании распространения световых лучей в сцене с учётом не только прямого освещения от источников света, но и многократных переотражений от поверхностей. За счёт этого удаётся учитывать глобальное освещение, то есть вклад света, который приходит в точку не только непосредственно от источника, но и после отражения от других объектов сцены.

В программе для каждого пикселя экрана из камеры испускаются лучи в направлении сцены. Если луч пересекает поверхность, вычисляется вклад собственного излучения поверхности, прямого освещения от источников света, а также вклад вторичных лучей, возникающих после отражения. Такой процесс повторяется рекурсивно до достижения заданной глубины трассировки или до прекращения пути.

2. Представление сцены

Сцена представляется набором треугольников, что соответствует условию задания. Каждый треугольник задаётся тремя вершинами в трёхмерном пространстве и материалом поверхности. Такой способ представления удобен тем, что позволяет как формировать сцену вручную, так и загружать её из внешнего OBJ-файла.

Для каждого треугольника вычисляются:

- геометрическая нормаль;

- площадь;
- возможность пересечения с лучом;
- случайная точка на поверхности, если треугольник используется как источник света.

Таким образом, треугольник в программе является базовым элементом геометрии сцены.

3. Цветовая модель

В программе используется цветовая модель RGB. Цвет каждой поверхности и интенсивность излучения источников задаются тремя компонентами: красной, зелёной и синей. Это соответствует требованиям задания.

Все вычисления производятся в вещественных значениях яркости. Это позволяет накапливать вклад большого числа лучей без потери точности. Уже после завершения рендеринга изображение переводится в диапазон отображения.

4. Модель материалов

Для описания оптических свойств поверхности используется упрощённая физически корректная модель, включающая две составляющие:

- диффузное отражение по закону Ламберта;
- зеркальное отражение по модели идеального зеркала.

Диффузная составляющая описывает матовые поверхности, рассеивающие свет во все направления полусферы над поверхностью. Зеркальная составляющая описывает отражение луча в строго определённом направлении, симметричном относительно нормали.

Для каждой поверхности могут одновременно присутствовать обе составляющие. В этом случае при моделировании выбирается одно из событий отражения:

- либо диффузное,
- либо зеркальное.

Выбор выполняется вероятностно, в зависимости от величины коэффициентов отражения. Такой подход соответствует использованию выборки по значимости и русской рулетки.

Дополнительно в программе введена проверка физичности материала: для каждого цветового канала выполняется условие

$$k_d + k_s \leq 1,$$

где k_d — коэффициент диффузного отражения, а k_s — коэффициент зеркального отражения.

Это гарантирует, что поверхность не отражает больше энергии, чем получает.

5. Алгоритм пересечения луча с треугольником

Для нахождения первой точки пересечения луча со сценой используется проверка пересечения луча с каждым треугольником сцены. Для конкретного треугольника применяется алгоритм на основе барицентрических координат, аналогичный алгоритму Мёллера–Трумбора.

Для каждого луча:

1. вычисляются векторы рёбер треугольника;
2. определяется, пересекает ли луч плоскость треугольника;
3. проверяется, лежит ли точка пересечения внутри треугольника;
4. из всех найденных пересечений выбирается ближайшее.

Результатом работы этого этапа являются:

- расстояние до точки пересечения;
- координаты точки;
- нормаль поверхности;
- материал пересечённого объекта.

6. Камера и генерация первичных лучей

В программе используется точечная камера, как требуется в задании. Положение камеры задаётся координатами точки наблюдения, а направление обзора — точкой, на которую направлена камера. Также задаётся вертикальный вектор и угол обзора.

Для каждого пикселя вычисляется направление луча, исходящего из камеры через соответствующую точку изображения. В результате для каждого пикселя формируется хотя бы один первичный луч.

7. Антиалиасинг

Для уменьшения ступенчатости границ объектов используется антиалиасинг методом случайной выборки внутри пикселя. Вместо прохождения луча строго через центр пикселя его координаты немного случайно смещаются в пределах площади пикселя. Это соответствует требованию задания.

Если для одного пикселя запускается несколько лучей, результаты усредняются. Благодаря этому границы объектов становятся менее резкими и уменьшается шум дискретизации.

8. Источники света

Источники света в программе моделируются как протяжённые светящиеся треугольники. Такой подход соответствует условию задания, где требуется использовать протяжённые источники света с ламбертовской диаграммой излучения.

Каждый источник имеет:

- геометрию;
- площадь;
- цвет излучения;
- интенсивность.

Для вычисления прямого освещения выбирается случайный источник света, а затем случайная точка на его поверхности. После этого строится теневой луч из рассматриваемой точки сцены к выбранной точке на источнике. Если на пути нет препятствий, вклад источника добавляется в итоговую яркость.

9. Прямое освещение

Прямое освещение вычисляется отдельно в точке пересечения луча с поверхностью. Для этого:

1. выбирается источник света;
2. на его поверхности выбирается случайная точка;
3. строится направление на источник;
4. проверяется, не перекрыт ли источник другими объектами;
5. вычисляется вклад освещения с учётом:
 - косинуса угла между нормалью поверхности и направлением на источник;
 - косинуса угла между нормалью источника и направлением на точку;
 - расстояния до источника;
 - функции отражения поверхности.

Этот этап позволяет учитывать освещение от источников непосредственно, без ожидания случайного попадания вторичного луча в источник.

10. Непрямое освещение и трассировка путей

Главная особенность метода трассировки путей — учёт непрямого освещения. После попадания луча в поверхность строится новый луч, направление которого зависит от выбранного события отражения:

- при диффузном отражении новое направление случайно выбирается в полусфере над поверхностью;
- при зеркальном отражении направление вычисляется по закону отражения.

Затем для нового луча снова выполняется поиск пересечения, вычисление освещения и возможное построение следующих лучей. Таким образом моделируется путь света в сцене.

Для ограничения времени вычислений используется максимальная глубина рекурсии. Если глубина достигнута, трассировка текущего пути прекращается.

11. Выбор события: выборка по значимости и русская рулетка

Так как поверхность может обладать и диффузной, и зеркальной составляющей, в программе используется вероятностный выбор события отражения. Вероятность диффузного или зеркального отражения определяется относительным вкладом соответствующей компоненты материала.

Такой подход позволяет:

- чаще выбирать более значимые события;
- уменьшать дисперсию оценки;
- сохранять корректность среднего результата.

Фактически это сочетание выборки по значимости и русской рулетки, как требуется в задании.

12. Прогрессивный рендеринг

Изображение строится прогрессивно. Это означает, что результат не вычисляется полностью за один проход, а накапливается постепенно. На каждом проходе для каждого пикселя добавляется новый случайный сэмпл, после чего изображение пересчитывается как среднее накопленных значений.

Преимущества такого подхода:

- можно наблюдать промежуточный результат уже в процессе рендеринга;
- пользователь может остановить вычисление, когда качество станет достаточным;
- на защите удобно демонстрировать постепенное улучшение изображения.

Программа поддерживает два режима завершения рендеринга:

- по заданному числу сэмплов на пиксель;
- по заданному лимиту времени.

13. Нормировка яркости и гамма-коррекция

После завершения рендеринга изображение существует в виде абсолютных яркостей. Для отображения его на экране и сохранения в файл необходимо перевести значения в диапазон $[0, 1]$, а затем в диапазон $[0, 255]$.

В программе используется следующая последовательность:

1. находится максимальная яркость в изображении;
2. все пиксели нормируются относительно этого значения;
3. значения, выходящие за пределы диапазона, ограничиваются;
4. применяется гамма-коррекция;
5. данные переводятся в 8-битный формат и сохраняются в файл PPM.

Гамма-коррекция необходима для корректного визуального отображения изображения на обычных устройствах вывода.

14. Пользовательский интерфейс программы

Программа содержит простой графический интерфейс на основе Tkinter, что соответствует требованию задания о наличии простого интерфейса для изменения параметров сцены и получения нового результата в процессе защиты.

Через интерфейс пользователь может изменять:

- разрешение изображения;
- число сэмплов на пиксель;
- максимальную глубину трассировки;
- параметры гамма-коррекции;
- режим остановки рендеринга;
- интенсивность и цвет источника света;
- положение камеры и точку, на которую она направлена;
- материал объекта;
- путь к OBJ-файлу.

Также в интерфейсе отображается:

- текущее состояние рендеринга;
- прогресс по строкам и проходам;
- промежуточное превью изображения.

15. Вывод по разделу

Таким образом, разработанная программа реализует основные требования задания: сцена задаётся треугольной геометрией, используется RGB-модель, учитываются диффузное и зеркальное отражение, выполняется трассировка путей с учётом глобального освещения, применяется выборка по значимости и русская рулетка, а итоговое изображение нормируется, подвергается гамма-коррекции и сохраняется в файл. Наличие простого графического интерфейса позволяет изменять параметры сцены и повторно запускать рендеринг, что делает программу удобной для демонстрации и защиты лабораторной работы.

Описание структуры программы и основных классов и функций

1. Общая структура программы

Программа состоит из нескольких логических частей:

- математический аппарат для работы с векторами и лучами;
- описание материалов и геометрических объектов сцены;
- описание камеры;
- алгоритмы трассировки лучей;
- функции постобработки изображения;
- формирование тестовой сцены;
- графический интерфейс пользователя.

2. Класс Vec3

Для работы с геометрией сцены и цветом в программе используется класс Vec3. Он применяется как для представления трёхмерных координат, так и для хранения RGB-компонент цвета. В классе реализованы основные векторные операции: сложение, вычитание, умножение, скалярное произведение, векторное произведение, вычисление длины, нормализация и ограничение значений по диапазону.

```
@dataclass
class Vec3:
    x: float
    y: float
    z: float

    def __add__(self, other):
        return Vec3(self.x + other.x, self.y + other.y, self.z + other.z)

    def __sub__(self, other):
```

```

        return Vec3(self.x - other.x, self.y - other.y, self.z - other.z)

    def __mul__(self, value):
        if isinstance(value, Vec3):
            return Vec3(self.x * value.x, self.y * value.y, self.z * value.z)
        return Vec3(self.x * value, self.y * value, self.z * value)

    def dot(self, other) -> float:
        return self.x * other.x + self.y * other.y + self.z * other.z

    def cross(self, other):
        return Vec3(
            self.y * other.z - self.z * other.y,
            self.z * other.x - self.x * other.z,
            self.x * other.y - self.y * other.x,
        )

    def normalized(self):
        l = self.length()
        if l < EPS:
            return Vec3(0.0, 0.0, 0.0)
        return self / l

```

Данный класс является базовым для всей программы. Через него выполняются практически все вычисления: построение лучей, вычисление нормалей, отражённых направлений, а также работа с цветами и яркостями.

3. Класс Ray

Луч в программе представлен классом Ray, который содержит точку начала и направление. Все вычисления трассировки сводятся к проверке пересечения таких лучей с объектами сцены.

```

@dataclass
class Ray:
    origin: Vec3
    direction: Vec3

```

Таким образом, класс Ray является минимальным, но необходимым элементом алгоритма трассировки путей.

4. Класс Material

Класс Material описывает оптические свойства поверхности. В нём задаются коэффициенты диффузного и зеркального отражения, а также собственное излучение поверхности. Кроме того, в этом классе реализована проверка физичности материала и выбор события отражения.

```
@dataclass
class Material:
    diffuse: Vec3
    specular: Vec3
    emission: Vec3 = field(default_factory=lambda: Vec3(0.0, 0.0, 0.0))
    name: str = "material"

    def validate_physical(self):
        for d, s, channel in zip(
            self.diffuse.tuple(),
            self.specular.tuple(),
            ("R", "G", "B")
        ):
            if d < 0 or s < 0:
                raise ValueError(f"Материал {self.name}: отрицательные
коэффициенты в канале {channel}.")
            if d + s > 1.0 + 1e-9:
                raise ValueError(
                    f"Материал {self.name}: нарушена физичность в канале
{channel}: "
                    f"diffuse({d:.3f}) + specular({s:.3f}) > 1."
                )

    def is_light(self) -> bool:
        return self.emission.max_component() > 0.0

    def choose_event(self) -> Tuple[str, Vec3, float]:
        pd = self.diffuse.max_component()
        ps = self.specular.max_component()
        s = pd + ps

        if s < EPS:
```

```

        return "absorb", Vec3(0.0, 0.0, 0.0), 1.0

    pd /= s
    ps /= s

    r = random.random()
    if r < pd:
        return "diffuse", self.diffuse, pd
    return "specular", self.specular, ps

```

Этот класс играет важную роль, так как именно в нём определяется, как поверхность взаимодействует со светом. Наличие метода `validate_physical()` позволяет контролировать корректность параметров материала, а метод `choose_event()` реализует выбор события по значимости и русскую рулетку.

5. Класс Triangle

Основным геометрическим примитивом сцены является треугольник. Класс `Triangle` хранит координаты трёх вершин и материал поверхности. В нём также реализованы методы для вычисления нормали, площади, пересечения с лучом и случайной выборки точки на поверхности.

```

@dataclass
class Triangle:
    a: Vec3
    b: Vec3
    c: Vec3
    material: Material

    def normal(self) -> Vec3:
        return (self.b - self.a).cross(self.c - self.a).normalized()

    def area(self) -> float:
        return 0.5 * (self.b - self.a).cross(self.c - self.a).length()

    def intersect(self, ray: Ray) -> Optional[Hit]:
        edge1 = self.b - self.a
        edge2 = self.c - self.a
        h = ray.direction.cross(edge2)

```

```

det = edge1.dot(h)

if abs(det) < EPS:
    return None

inv_det = 1.0 / det
s = ray.origin - self.a
u = inv_det * s.dot(h)
if u < 0.0 or u > 1.0:
    return None

q = s.cross(edge1)
v = inv_det * ray.direction.dot(q)
if v < 0.0 or u + v > 1.0:
    return None

t = inv_det * edge2.dot(q)
if t < EPS:
    return None

pos = ray.origin + ray.direction * t
n = self.normal()
if n.dot(ray.direction) > 0.0:
    n = n * -1.0

return Hit(t=t, position=pos, normal=n, material=self.material)

```

Метод `intersect()` является одним из ключевых в программе, поскольку именно он позволяет определить, попадает ли луч в треугольник. Без этого этапа невозможно вычислить освещение, отражение и видимость объектов.

6. Класс Camera

Для построения изображения используется точечная камера. Класс Camera хранит положение камеры, точку, на которую она направлена, вектор вверх и угол обзора. Основная функция этого класса - генерация лучей для пикселей изображения.

```
@dataclass
class Camera:
    position: Vec3
    target: Vec3
    up: Vec3
    fov_deg: float

    def generate_ray(self, x: float, y: float, width: int, height: int) -> Ray:
        forward = (self.target - self.position).normalized()
        right = forward.cross(self.up).normalized()
        true_up = right.cross(forward).normalized()

        aspect = width / height
        scale = math.tan(math.radians(self.fov_deg) * 0.5)

        px = (2.0 * ((x + 0.5) / width) - 1.0) * aspect * scale
        py = (1.0 - 2.0 * ((y + 0.5) / height)) * scale

        direction = (forward + right * px + true_up * py).normalized()
        return Ray(self.position, direction)
```

Таким образом, класс Camera связывает двумерную плоскость изображения с трёхмерной сценой и позволяет запускать первичные лучи для дальнейшей трассировки.

7. Класс Scene

Класс Scene объединяет все треугольники сцены и отдельно хранит список источников света. В нём реализованы методы добавления объектов, поиска ближайшего пересечения и проверки перекрытия луча другими объектами.

```
class Scene:

    def __init__(self):
        self.triangles: List[Triangle] = []
        self.lights: List[Triangle] = []

    def add_triangle(self, tri: Triangle):
        tri.material.validate_physical()
        self.triangles.append(tri)
        if tri.material.is_light():
            self.lights.append(tri)

    def intersect(self, ray: Ray) -> Optional[Hit]:
        closest_t = INF
        closest_hit = None
        for tri in self.triangles:
            hit = tri.intersect(ray)
            if hit and hit.t < closest_t:
                closest_t = hit.t
                closest_hit = hit
        return closest_hit

    def occluded(self, ray: Ray, max_dist: float) -> bool:
        for tri in self.triangles:
            hit = tri.intersect(ray)
            if hit and hit.t < max_dist - EPS:
                return True
        return False
```

Класс Scene можно рассматривать как контейнер, который предоставляет трассировщику доступ ко всем объектам сцены и позволяет выполнять базовые операции над всей геометрией.

8. Класс PathTracer

Основная логика формирования изображения сосредоточена в классе PathTracer. В нём реализованы вычисление прямого освещения, рекурсивная трассировка путей и прогрессивный рендеринг изображения.

8.1. Вычисление прямого освещения

Метод `estimate_direct_light()` отвечает за вклад света от протяжённого источника. Он выбирает источник света, случайную точку на его поверхности, строит теневой луч и, если источник видим, добавляет его вклад в освещённость.

```
def estimate_direct_light(self, hit: Hit) -> Vec3:
    picked = self.scene.choose_light()
    if picked is None:
        return Vec3(0.0, 0.0, 0.0)

    light_tri, p_light_select = picked
    lp, ln, pdf_area = light_tri.sample_point()

    to_light = lp - hit.position
    dist2 = to_light.dot(to_light)
    dist = math.sqrt(dist2)
    wi = to_light / max(EPS, dist)

    cos_surface = max(0.0, hit.normal.dot(wi))
    cos_light = max(0.0, ln.dot(wi * -1.0))

    if cos_surface <= 0.0 or cos_light <= 0.0:
        return Vec3(0.0, 0.0, 0.0)

    shadow_ray = Ray(hit.position + hit.normal * 1e-4, wi)
    if self.scene.occluded(shadow_ray, dist):
        return Vec3(0.0, 0.0, 0.0)

    Le = light_tri.material.emission
    brdf = hit.material.diffuse / math.pi
    pdf = max(EPS, p_light_select * pdf_area)

    return brdf * Le * (cos_surface * cos_light / max(EPS, dist2)) / pdf
```

Этот метод позволяет учитывать прямое освещение более эффективно, чем при полностью случайной трассировке без отдельной выборки источника света.

8.2. Рекурсивная трассировка

Метод `trace()` является центральной частью алгоритма. Он определяет точку пересечения луча со сценой, учитывает собственное излучение поверхности, выбирает событие отражения и запускает вторичный луч.

```
def trace(self, ray: Ray, depth: int = 0) -> Vec3:
    if depth >= self.max_depth:
        return Vec3(0.0, 0.0, 0.0)

    hit = self.scene.intersect(ray)
    if hit is None:
        return self.background

    emitted = hit.material.emission
    event, color, p_event = hit.material.choose_event()

    if event == "absorb":
        return emitted

    result = emitted

    if event == "diffuse":
        direct = self.estimate_direct_light(hit)

        new_dir, pdf_dir = cosine_sample_hemisphere(hit.normal)
        brdf = hit.material.diffuse / math.pi
        cos_theta = max(0.0, hit.normal.dot(new_dir))

        new_ray = Ray(hit.position + hit.normal * 1e-4, new_dir)
        incoming = self.trace(new_ray, depth + 1)

        indirect = brdf * incoming * (cos_theta / max(EPS, pdf_dir))
        result += (direct + indirect) / max(EPS, p_event)

    elif event == "specular":
```

```

        new_dir = reflect(ray.direction, hit.normal).normalized()
        new_ray = Ray(hit.position + hit.normal * 1e-4, new_dir)
        incoming = self.trace(new_ray, depth + 1)
        result += (color * incoming) / max(EPS, p_event)

    return result

```

Именно этот метод реализует учёт непрямого освещения, то есть многократных отражений света в сцене.

8.3. Прогрессивный рендеринг

Метод `render_progressive()` выполняет рендеринг по проходам. На каждом проходе добавляется новый случайный вклад в каждый пиксель, после чего вычисляется промежуточное среднее изображение.

```

def render_progressive(
    self,
    width: int,
    height: int,
    target_spp: int,
    time_limit_sec: float,
    progress_callback=None,
    preview_callback=None
) -> List[List[Vec3]]:
    accum = [[Vec3(0.0, 0.0, 0.0) for _ in range(width)] for _ in range(height)]
    passes_done = 0
    start_time = time.time()

    while not self.stop_flag:
        if 0 < target_spp <= passes_done:
            break

        if 0 < time_limit_sec <= (time.time() - start_time):
            break

        for y in range(height):
            if self.stop_flag:
                break

            for x in range(width):

```

```

        jx = random.random() - 0.5
        jy = random.random() - 0.5
        ray = self.camera.generate_ray(x + jx, y + jy, width, height)
        sample = self.trace(ray, 0)
        accum[y][x] = accum[y][x] + sample

    passes_done += 1

```

Такой подход позволяет постепенно улучшать качество изображения и выводить промежуточный результат в интерфейс программы.

9. Функции постобработки

После завершения рендеринга изображение должно быть приведено к диапазону отображения. Для этого используется функция `tonemap_and_gamma()`, которая выполняет нормировку, ограничение диапазона и гамма-коррекцию.

```

def tonemap_and_gamma(framebuffer: List[List[Vec3]], gamma: float = 2.2) ->
List[List[Tuple[int, int, int]]]:
    max_lum = 0.0
    for row in framebuffer:
        for c in row:
            max_lum = max(max_lum, c.max_component())

    scale = 1.0 / max(EPS, max_lum)

    def to_byte(value: float) -> int:
        return int(round(max(0.0, min(1.0, value)) * 255.0))

    img = []
    inv_gamma = 1.0 / gamma
    for row in framebuffer:
        out_row = []
        for c in row:
            mapped = (c * scale).clamp(0.0, 1.0)
            mapped = Vec3(
                mapped.x ** inv_gamma,
                mapped.y ** inv_gamma,
                mapped.z ** inv_gamma,

```

```

        )
        out_row.append((
            to_byte(mapped.x),
            to_byte(mapped.y),
            to_byte(mapped.z),
        ))
        img.append(out_row)

    return img

```

После этого готовое изображение сохраняется в файл формата PPM.

10. Формирование сцены

Для демонстрации работы программы используется функция `make_default_scene()`, которая создаёт тестовую сцену из треугольников: стены, пол, потолок, объект в центре и протяжённый источник света.

```

def make_default_scene(
    include_obj_path: Optional[str],
    light_intensity: float,
    light_color: Vec3,
    object_material_mode: str
) -> Tuple[Scene, Camera]:
    scene = Scene()

    white = Material(diffuse=Vec3(0.75, 0.75, 0.75), specular=Vec3(0.0, 0.0,
0.0), name="white")

    red = Material(diffuse=Vec3(0.75, 0.2, 0.2), specular=Vec3(0.0, 0.0, 0.0),
name="red")

    green = Material(diffuse=Vec3(0.2, 0.75, 0.2), specular=Vec3(0.0, 0.0, 0.0),
name="green")

```

В этой функции также задаются параметры камеры и, при необходимости, подключается дополнительная геометрия из OBJ-файла.

11. Графический интерфейс

Пользовательский интерфейс реализован в классе PathTracerApp. Он отвечает за:

- ввод параметров рендера;
- изменение параметров сцены;
- запуск и остановку рендеринга;
- отображение статуса и промежуточного результата;
- сохранение изображения.

Пример создания элементов интерфейса:

```
ttk.Label(left, text="Параметры рендера", font=("Arial", 12,
"bold")).pack(anchor="w", pady=(0, 8))

_add_labeled_entry(left, "Ширина (не меньше 500):", self.width_var)
_add_labeled_entry(left, "Высота (не меньше 500):", self.height_var)
_add_labeled_entry(left, "SPP:", self.spp_var)
_add_labeled_entry(left, "Макс. глубина:", self.depth_var)
_add_labeled_entry(left, "Гамма:", self.gamma_var)
```

Кроме того, в интерфейсе реализована прокрутка панели параметров, что позволяет использовать большое число настраиваемых параметров даже при ограниченной высоте окна.

12. Вывод по разделу

Таким образом, структура программы построена по модульному принципу. Отдельно выделены математические операции, описание сцены и материалов, логика трассировки лучей, постобработка и пользовательский интерфейс. Это делает программу более понятной с точки зрения анализа, а также позволяет подробно объяснить назначение каждого фрагмента кода в отчёте и при защите лабораторной работы.

Подробное пояснение ключевых фрагментов программы

1. Пересечение луча с треугольником

Одним из самых важных этапов трассировки путей является определение того, пересекает ли луч какой-либо объект сцены. Так как сцена представлена треугольниками, для каждого луча необходимо проверить пересечение с каждым треугольником. В программе для этого используется метод `intersect()` класса `Triangle`.

```
def intersect(self, ray: Ray) -> Optional[Hit]:
    edge1 = self.b - self.a
    edge2 = self.c - self.a
    h = ray.direction.cross(edge2)
    det = edge1.dot(h)

    if abs(det) < EPS:
        return None

    inv_det = 1.0 / det
    s = ray.origin - self.a
    u = inv_det * s.dot(h)
    if u < 0.0 or u > 1.0:
        return None

    q = s.cross(edge1)
    v = inv_det * ray.direction.dot(q)
    if v < 0.0 or u + v > 1.0:
        return None

    t = inv_det * edge2.dot(q)
    if t < EPS:
        return None

    pos = ray.origin + ray.direction * t
    n = self.normal()
    if n.dot(ray.direction) > 0.0:
        n = n * -1.0

    return Hit(t=t, position=pos, normal=n, material=self.material)
```

В данном фрагменте сначала вычисляются два ребра треугольника. Затем проверяется, пересекает ли луч плоскость треугольника. После этого вычисляются барицентрические координаты u и v , по которым определяется, находится ли точка пересечения внутри треугольника. Если пересечение найдено, формируется объект `Hit`, содержащий расстояние до точки, координаты точки пересечения, нормаль и материал поверхности.

2. Выбор события отражения

Так как поверхность может иметь одновременно диффузную и зеркальную составляющие, необходимо выбрать, какой тип отражения будет смоделирован для текущего луча. В программе это делается вероятностным способом с использованием выборки по значимости и русской рулетки.

```
def choose_event(self) -> Tuple[str, Vec3, float]:
    pd = self.diffuse.max_component()
    ps = self.specular.max_component()
    s = pd + ps

    if s < EPS:
        return "absorb", Vec3(0.0, 0.0, 0.0), 1.0

    pd /= s
    ps /= s

    r = random.random()
    if r < pd:
        return "diffuse", self.diffuse, pd
    return "specular", self.specular, ps
```

В этом методе сначала оцениваются относительные вклады диффузной и зеркальной компонент. Затем они нормируются до вероятностей. После генерации случайного числа выбирается одно из двух событий: `diffuse` или `specular`. Если поверхность практически не отражает свет, выбирается состояние `absorb`.

Такой подход позволяет чаще выбирать более значимое событие и тем самым уменьшать шум в изображении.

3. Проверка физичности материала

Так как в задании требуется физически корректная модель, в программе введена проверка, чтобы сумма коэффициентов отражения по каждому цветовому каналу не превышала единицу.

```
def validate_physical(self):
    for d, s, channel in zip(
        self.diffuse.tuple(),
        self.specular.tuple(),
        ("R", "G", "B")
    ):
        if d < 0 or s < 0:
            raise ValueError(f"Материал {self.name}: отрицательные коэффициенты в канале {channel}.")
        if d + s > 1.0 + 1e-9:
            raise ValueError(
                f"Материал {self.name}: нарушена физичность в канале {channel}: "
                f"diffuse({d:.3f}) + specular({s:.3f}) > 1."
            )
```

В этом коде для каждого цветового канала выполняются две проверки:

- коэффициенты отражения не должны быть отрицательными;
- сумма диффузной и зеркальной компонент не должна превышать 1.

Если условие нарушено, программа выдаёт сообщение об ошибке. Это защищает сцену от некорректных материалов и помогает соблюдать закон сохранения энергии в упрощённой модели отражения.

4. Генерация первичного луча камеры

Для каждого пикселя изображения необходимо построить луч, исходящий из камеры в сцену. Этот процесс реализуется в методе `generate_ray()` класса `Camera`.

```
def generate_ray(self, x: float, y: float, width: int, height: int) -> Ray:
    forward = (self.target - self.position).normalized()
    right = forward.cross(self.up).normalized()
    true_up = right.cross(forward).normalized()

    aspect = width / height
    scale = math.tan(math.radians(self.fov_deg) * 0.5)
```

```
px = (2.0 * ((x + 0.5) / width) - 1.0) * aspect * scale
py = (1.0 - 2.0 * ((y + 0.5) / height)) * scale

direction = (forward + right * px + true_up * py).normalized()

return Ray(self.position, direction)
```

В данном коде сначала строится базис камеры: направление вперёд, вправо и вверх. Затем координаты пикселя переводятся в координаты на плоскости изображения с учётом соотношения сторон и угла обзора. После этого вычисляется итоговое направление луча.

Таким образом, каждый пиксель связывается с определённым направлением обзора в трёхмерной сцене.

5. Антиалиасинг через случайную выборку внутри пикселя

Чтобы уменьшить ступенчатость границ и сделать оценку более корректной, в программе используется антиалиасинг. Для этого координаты луча случайно смещаются в пределах пикселя.

```
for x in range(width):
    jx = random.random() - 0.5
    jy = random.random() - 0.5
    ray = self.camera.generate_ray(x + jx, y + jy, width, height)
    sample = self.trace(ray, 0)
    accum[y][x] = accum[y][x] + sample
```

Здесь jx и jy - случайные смещения внутри текущего пикселя. Благодаря этому лучи не проходят всегда через одну и ту же точку, а покрывают внутреннюю область пикселя случайными выборками.

После нескольких проходов результаты усредняются, и это уменьшает ступенчатость на границах объектов и делает итоговое изображение более естественным.

6. Выбор случайного направления при диффузном отражении

Если на поверхности выбрано диффузное событие, необходимо сгенерировать новое направление луча в верхней полусфере относительно нормали. В программе используется косинусная выборка, так как она хорошо согласуется с ламбертовской моделью отражения.

```
def cosine_sample_hemisphere(normal: Vec3) -> Tuple[Vec3, float]:  
    r1 = random.random()  
    r2 = random.random()  
    phi = 2.0 * math.pi * r1  
    r = math.sqrt(r2)  
  
    x = r * math.cos(phi)  
    y = r * math.sin(phi)  
    z = math.sqrt(max(0.0, 1.0 - r2))  
  
    t, b = build_orthonormal_basis(normal)  
    direction = (t * x + b * y + normal * z).normalized()  
    pdf = max(EPS, direction.dot(normal)) / math.pi  
    return direction, pdf
```

В этом фрагменте сначала генерируется случайная точка в локальной системе координат полусферы. Затем через ортонормированный базис она переводится в мировую систему координат. Также вычисляется плотность вероятности pdf, которая необходима для корректной оценки вклада луча.

Использование косинусной выборки уменьшает дисперсию по сравнению с равномерной выборкой по полусфере.

7. Вычисление прямого освещения

Для уменьшения шума в изображении отдельно вычисляется вклад прямого света от источников. Это особенно важно, поскольку полностью случайное попадание вторичных лучей в источник света происходит редко.

```
def estimate_direct_light(self, hit: Hit) -> Vec3:

    picked = self.scene.choose_light()

    if picked is None:
        return Vec3(0.0, 0.0, 0.0)

    light_tri, p_light_select = picked
    lp, ln, pdf_area = light_tri.sample_point()

    to_light = lp - hit.position
    dist2 = to_light.dot(to_light)
    dist = math.sqrt(dist2)
    wi = to_light / max(EPS, dist)

    cos_surface = max(0.0, hit.normal.dot(wi))
    cos_light = max(0.0, ln.dot(wi * -1.0))

    if cos_surface <= 0.0 or cos_light <= 0.0:
        return Vec3(0.0, 0.0, 0.0)

    shadow_ray = Ray(hit.position + hit.normal * 1e-4, wi)
    if self.scene.occluded(shadow_ray, dist):
        return Vec3(0.0, 0.0, 0.0)

    Le = light_tri.material.emission
    brdf = hit.material.diffuse / math.pi
    pdf = max(EPS, p_light_select * pdf_area)

    return brdf * Le * (cos_surface * cos_light / max(EPS, dist2)) / pdf
```

В данном коде выбирается случайный источник света и случайная точка на его поверхности. Затем строится направление на эту точку и проверяется, не перекрыт ли путь другим объектом. Если источник видим, вычисляется его вклад с учётом геометрических факторов, расстояния и BRDF поверхности.

Таким образом, прямой свет учитывается явно, что делает изображение более стабильным и уменьшает шум.

8. Рекурсивное вычисление полного освещения

Основной вклад в изображение формируется в методе `trace()`. Он объединяет собственное излучение, прямое освещение и вклад вторичных лучей.

```
def trace(self, ray: Ray, depth: int = 0) -> Vec3:
    if depth >= self.max_depth:
        return Vec3(0.0, 0.0, 0.0)

    hit = self.scene.intersect(ray)
    if hit is None:
        return self.background

    emitted = hit.material.emission
    event, color, p_event = hit.material.choose_event()

    if event == "absorb":
        return emitted

    result = emitted

    if event == "diffuse":
        direct = self.estimate_direct_light(hit)

        new_dir, pdf_dir = cosine_sample_hemisphere(hit.normal)
        brdf = hit.material.diffuse / math.pi
        cos_theta = max(0.0, hit.normal.dot(new_dir))

        new_ray = Ray(hit.position + hit.normal * 1e-4, new_dir)
        incoming = self.trace(new_ray, depth + 1)

        indirect = brdf * incoming * (cos_theta / max(EPS, pdf_dir))
        result += (direct + indirect) / max(EPS, p_event)
```

```

elif event == "specular":
    new_dir = reflect(ray.direction, hit.normal).normalized()
    new_ray = Ray(hit.position + hit.normal * 1e-4, new_dir)
    incoming = self.trace(new_ray, depth + 1)
    result += (color * incoming) / max(EPS, p_event)

return result

```

Сначала в этом методе проверяется, не достигнута ли максимальная глубина трассировки. Затем находится пересечение луча со сценой. Если пересечения нет, возвращается цвет фона. Если пересечение найдено, учитывается собственное излучение поверхности и выбирается тип отражения.

Для диффузного отражения дополнительно учитываются:

- прямой свет от источника;
- вклад случайно выбранного вторичного луча.

Для зеркального отражения строится отражённый луч по закону зеркального отражения.

Этот метод является центральным в программе, так как именно он моделирует многократное распространение света в сцене.

9. Прогрессивное накопление изображения

Рендеринг изображения выполняется по проходам. На каждом проходе для всех пикселей добавляется по одному новому сэмплу, а затем изображение улучшается постепенно.

```

def render_progressive(
    self,
    width: int,
    height: int,
    target_spp: int,
    time_limit_sec: float,
    progress_callback=None,
    preview_callback=None
) -> List[List[Vec3]]:
    accum = [[Vec3(0.0, 0.0, 0.0) for _ in range(width)] for _ in range(height)]
    passes_done = 0
    start_time = time.time()

```

```

while not self.stop_flag:
    if 0 < target_spp <= passes_done:
        break

    if 0 < time_limit_sec <= (time.time() - start_time):
        break

    for y in range(height):
        if self.stop_flag:
            break

        for x in range(width):
            jx = random.random() - 0.5
            jy = random.random() - 0.5
            ray = self.camera.generate_ray(x + jx, y + jy, width, height)
            sample = self.trace(ray, 0)
            accum[y][x] = accum[y][x] + sample

    passes_done += 1

```

В этом фрагменте массив `accum` хранит накопленную сумму вкладов для каждого пикселя. После каждого прохода число проходов увеличивается, а итоговое изображение в любой момент может быть получено как среднее значение накопленных сэмплов.

Преимущество такого подхода состоит в том, что изображение можно наблюдать уже в процессе вычислений, а качество будет постепенно улучшаться.

10. Нормировка и гамма-коррекция

После завершения трассировки изображение всё ещё хранится в виде вещественных значений яркости. Для записи в файл нужно привести их к диапазону 0...255.

```

def tonemap_and_gamma(framebuffer: List[List[Vec3]], gamma: float = 2.2) -> List[List[Tuple[int, int, int]]]:
    max_lum = 0.0

    for row in framebuffer:
        for c in row:
            max_lum = max(max_lum, c.max_component())

    scale = 1.0 / max(EPS, max_lum)

```

```

def to_byte(value: float) -> int:
    return int(round(max(0.0, min(1.0, value)) * 255.0))

img = []
inv_gamma = 1.0 / gamma
for row in framebuffer:
    out_row = []
    for c in row:
        mapped = (c * scale).clamp(0.0, 1.0)
        mapped = Vec3(
            mapped.x ** inv_gamma,
            mapped.y ** inv_gamma,
            mapped.z ** inv_gamma,
        )
        out_row.append((
            to_byte(mapped.x),
            to_byte(mapped.y),
            to_byte(mapped.z),
        ))
    img.append(out_row)

return img

```

В этом коде сначала ищется максимальная яркость в изображении. Затем все пиксели нормируются относительно неё. После этого применяется гамма-коррекция и значения переводятся в диапазон байта.

Этот этап необходим, чтобы итоговое изображение можно было корректно сохранить и просматривать на обычных устройствах.

11. Вывод по разделу

Рассмотренные фрагменты программы показывают основные этапы формирования изображения методом трассировки путей: пересечение лучей с объектами, выбор типа отражения, вычисление прямого и непрямого освещения, прогрессивное накопление результата и постобработку итогового изображения. Приведённые участки кода являются ключевыми для понимания работы программы и подтверждают, что реализованный алгоритм соответствует поставленной задаче.

Тестирование программы и примеры полученных результатов

В ходе тестирования были проверены основные возможности разработанной программы: базовое формирование изображения сцены, изменение внешнего вида объекта при выборе разных материалов, изменение освещения сцены при настройке источника света, а также прогрессивное улучшение качества изображения по мере накопления проходов.

1. Базовый тест рендеринга

Для проверки общей работоспособности программы был выполнен рендер встроенной сцены при стандартных параметрах. Использовалось разрешение 500×500 пикселей, число сэмплов на пиксель 250, максимальная глубина трассировки 4, материал объекта `diffuse_gray`, интенсивность света 12.0 и тёплый цвет источника света.

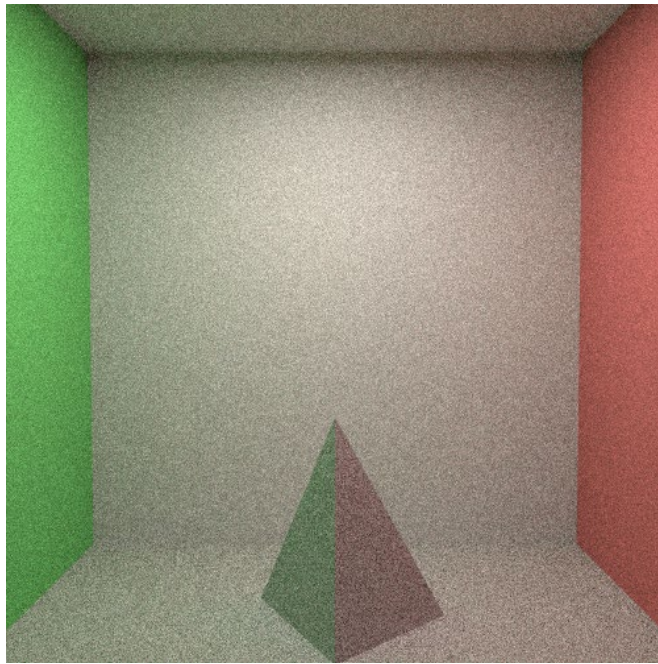


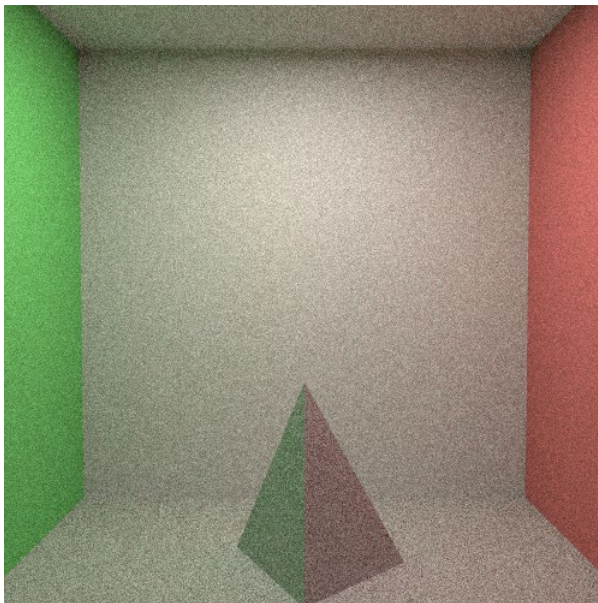
Рисунок 1 – Результат базового рендеринга сцены

Вывод: программа корректно формирует изображение трёхмерной сцены и сохраняет полученный результат.

2. Тест изменения материала объекта

Для проверки влияния материала на результат рендеринга были выполнены два запуска программы с одинаковыми параметрами сцены, но с разными материалами объекта: `diffuse_gray` и `mirror`.

а)



б)

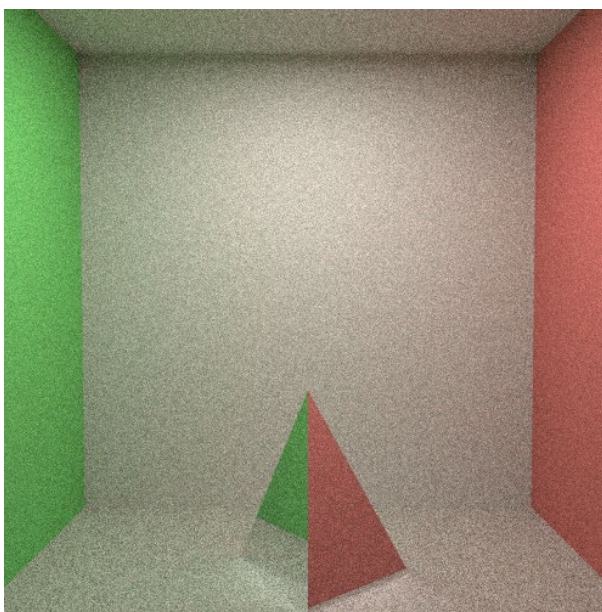


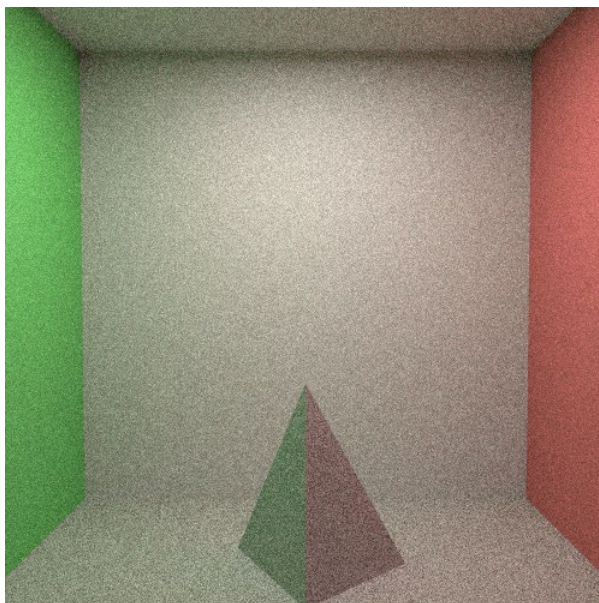
Рисунок 2 – Сравнение результатов рендеринга при разных материалах объекта: а — `diffuse_gray`, б — `mirror`

Вывод: изменение материала приводит к заметному изменению внешнего вида объекта и характера отражения света.

3. Тест изменения параметров освещения

Для проверки влияния источника света на итоговое изображение были выполнены два запуска программы с разным цветом освещения: тёплым и холодным.

а)



б)

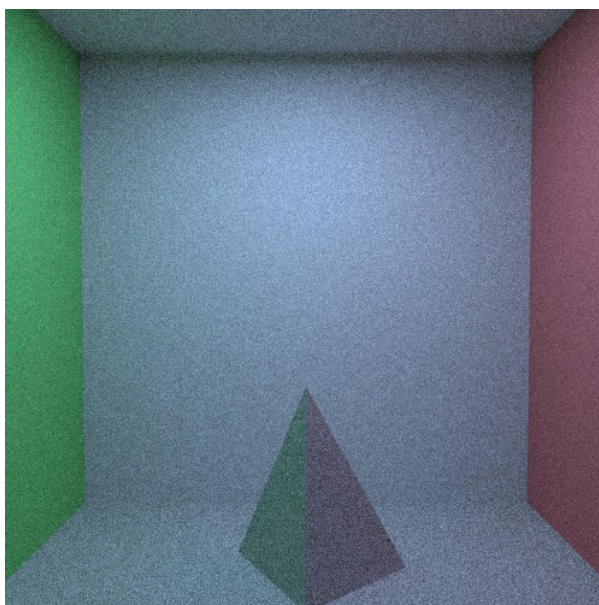


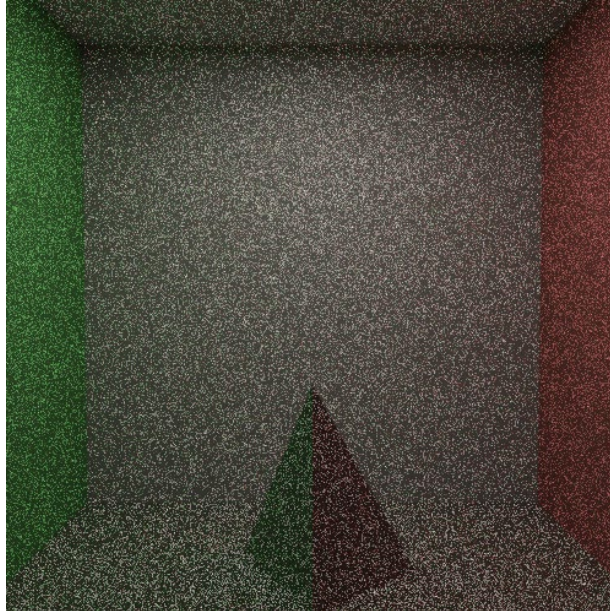
Рисунок 3 – Сравнение результатов рендеринга при разных параметрах освещения: а — тёплый свет, б — холодный свет

Вывод: изменение параметров источника света корректно влияет на яркость и цветовой тон изображения.

4. Тест прогрессивного рендеринга

Для проверки прогрессивного режима были сопоставлены промежуточное и более позднее состояние изображения. При увеличении числа проходов уровень шума снижался, а детали сцены становились более различимыми.

а)



б)

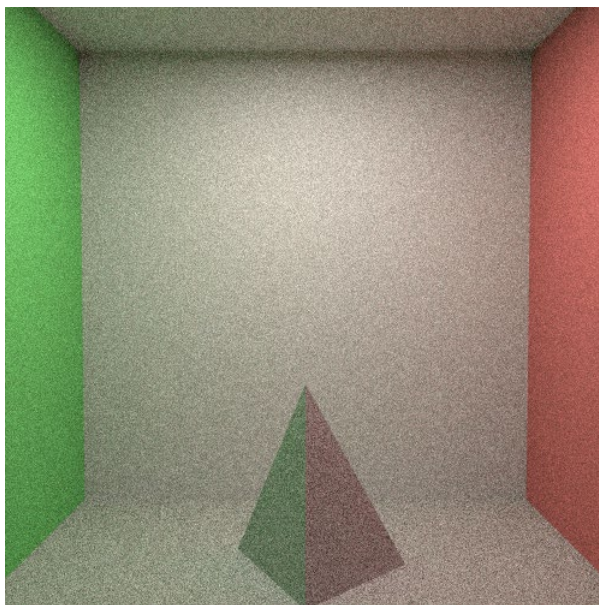


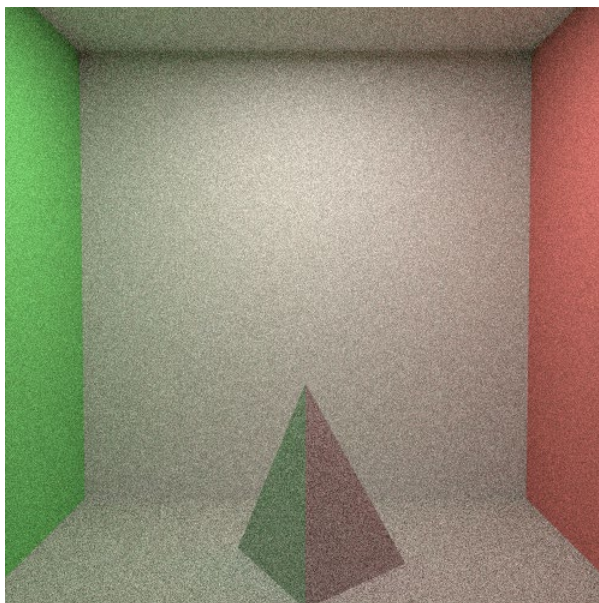
Рисунок 4 – Прогрессивное улучшение качества изображения: а — ранний этап рендеринга, б — более поздний этап

Вывод: программа корректно реализует прогрессивное накопление результата, что позволяет наблюдать постепенное улучшение качества изображения.

5. Тест изменения положения камеры

Дополнительно был проведён тест, направленный на проверку корректности формирования изображения при изменении положения камеры.

а)



б)

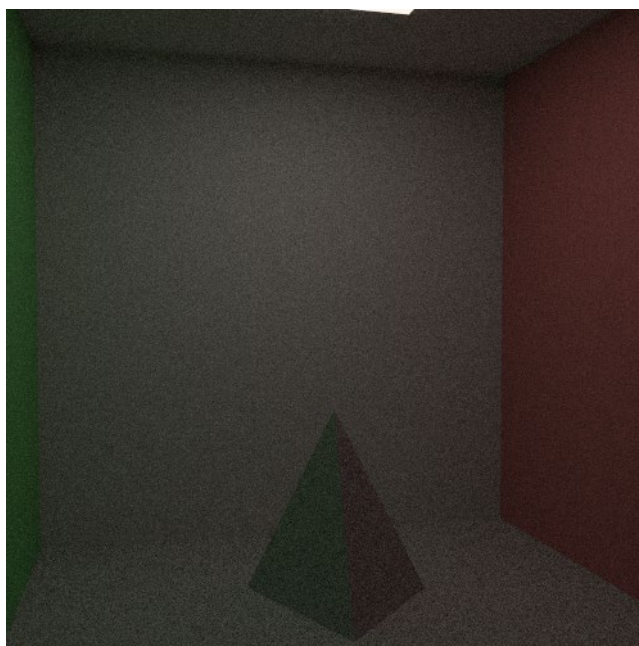


Рисунок 5 – Изменение изображения сцены при изменении положения камеры: а — базовый ракурс, б — боковой ракурс

Вывод: изменение положения камеры приводит к изменению ракурса сцены и подтверждает корректную работу модели камеры и генерации первичных лучей.

Заключение

Требования задания, связанные с треугольной геометрией сцены, RGB-моделью, физичностью материалов, антиалиасингом, протяжённым источником света и сохранением результата в файл, были учтены в реализованной программе и отражены в соответствующих разделах отчёта.

Также был разработан простой графический интерфейс на основе Tkinter, позволяющий изменять основные параметры рендеринга и сцены: разрешение изображения, число проходов, глубину трассировки, параметры источника света, положение камеры, материал объекта и режим остановки рендеринга. Это обеспечивает удобство демонстрации программы в процессе защиты и соответствует требованию задания о наличии простого интерфейса для изменения параметров сцены и повторного получения результата. В отчёте уже приведены разделы с описанием алгоритмов, структуры программы, ключевых фрагментов кода и тестирования, что подтверждает понимание реализованного решения.

В процессе тестирования было подтверждено, что программа корректно формирует изображение сцены, позволяет изменять внешний вид объекта при смене материала, корректно реагирует на изменение параметров освещения и камеры, а также реализует прогрессивное улучшение качества изображения по мере накопления проходов. Таким образом, цель лабораторной работы была достигнута: были освоены основные принципы синтеза изображений трёхмерных сцен с учётом глобального освещения методом трассировки путей, а также получены практические навыки построения собственного простого рендерера и пользовательского интерфейса для управления его параметрами.